SOFTWARE ENGINEERING PROJECT REPORT

Smart Lab Management & Continuous Evaluation System

A Centralized Platform for Laboratory Records and Academic Evaluation

LABORATORY RECORDS • TEAM MANAGEMENT
Attendance • Evaluation • Reports

PREPARED BY

TEAM ARISE

Madhav Gupta • Pratham Verma • Dishant Kumar

Course: Software Engineering

Institution: [THAPAR INSTITUTE OF ENGINEERING AND TECHNOLOGY]

Faculty / Project Guide: [Dr. Raghav B. Venkataramaiyer]

Academic Session: [2026]

October 2026

Declaration

We, the members of Team Arise, declare that this report titled *Smart Lab Management and Continuous Evaluation System* has been prepared for academic purposes to document the design, structure, and intended functionality of our project. The report distinguishes between the current project structure and proposed future enhancements. Implementation claims and test outcomes must be checked against the final running application before submission. Any institution-specific declaration required by the course should be reviewed with the project guide.

Project Team: Team Arise **Project:** Software Engineering

Madhav Gupta
Team Member

Pratham Verma
Team Member

Dishant Kumar
Team Member

Acknowledgement

We would like to thank our project guide and faculty for their guidance and feedback during the development of this project. We also acknowledge the open-source technologies and documentation that support the development of Flutter applications and Firebase-based systems.

Abstract

Laboratory courses require teachers to maintain student records, teams, groups, laboratory activities, attendance, marks, feedback, and reports. When these records are managed through separate spreadsheets and manual documents, the process can become repetitive, fragmented, and difficult to maintain. This project, the *Smart Lab Management and Continuous Evaluation System*, aims to provide a centralized digital platform for organizing laboratory-related academic information.

The application is developed using Flutter and Dart, with Firebase and Cloud Firestore forming the cloud data layer. The project structure includes screens and services associated with student, team, group, laboratory activity, attendance, evaluation, report, and Excel-import workflows. Excel-based data ingestion is intended to reduce repeated manual entry by processing structured student and team information. The modular organization separates user-interface screens, data models, and service operations.

This report describes the problem, objectives, scope, requirements, proposed architecture, data model, technology choices, key modules, limitations, and future enhancements. The current project archive contains files for several of the described modules; however, the existence of a screen or service file alone does not prove that every workflow is fully implemented or tested. Therefore, implementation and test status should be verified against the running application before final academic submission. Future enhancements include a student portal, online submissions, notifications, analytics, QR-based attendance, and AI-assisted evaluation.

Keywords: Laboratory management, continuous evaluation, Flutter, Dart, Firebase, Cloud Firestore, Excel import, attendance management.

Contents

Declaration **i**

1 Introduction **1**

1.1 Background . . . 1

1.2 Project Overview . . . 1

1.3 Motivation . . . 1

1.4 Report Organization . . . 2

2 Problem Statement and Objectives **3**

2.1 Problem Statement . . . 3

2.2 Problems in the Existing Approach . . . 3

2.3 Aim . . . 4

2.4 Objectives . . . 4

2.5 Proposed Solution . . . 4

3 Scope and Requirements **5**

3.1 Project Scope . . . 5

3.1.1 Current Project Focus . . . 5

3.1.2 Out of Scope for the Current Version . . . 5

3.2 Stakeholders . . . 6

3.3 Functional Requirements . . . 6

3.4 Non-Functional Requirements . . . 7

3.5 Assumptions and Constraints . . . 7

4 System Design **9**

4.1 High-Level Architecture . . . 9

4.2 Application Workflow . . . 10

4.3 Major Components 10
4.4 Conceptual Data Model 10
4.5 Data Entities 11

5 Technology Stack and Module Design 12

5.1 Flutter and Dart 12
5.2 Firebase and Cloud Firestore 12
5.3 Excel-Based Data Import 12
5.4 Module Descriptions 13
5.5 Software Organization 13

6 Implementation Overview 15

6.1 Application Entry Point and Configuration 15
6.2 Service Layer 15
6.3 Record Management 15
6.4 Excel Import Flow 15
6.5 Attendance and Evaluation 16
6.6 Screenshots 16

7 Verification and Test Plan 18

7.1 Purpose of Testing 18
7.2 Testing Categories 19
7.3 Evidence to Include 19

8 Current Limitations and Future Enhancements 20

8.1 Current Limitations 20
8.2 Future Enhancements 20
8.2.1 Student Portal 20
8.2.2 Online Submission Management 21
8.2.3 QR-Based Attendance 21
8.2.4 Analytics Dashboard 21

8.2.5 Notifications 21
8.2.6 AI-Assisted Evaluation 21
8.2.7 Access Control and Security 21

9 Conclusion **22**

References **23**

A Final Submission Checklist **24**

List of Tables

3.1 Project stakeholders and interests 6
3.2 Functional requirements 6
3.3 Non-functional requirements 7

4.1 Major application components 10
4.2 Conceptual entities and example attributes 11

5.1 Project modules 13

7.1 Proposed test cases 18

List of Figures

4.1 Conceptual architecture of the application. Confirm exact call paths against the implementation. 9

4.2 Conceptual teacher workflow. The exact sequence may vary by screen and implementation. 10

4.3 Conceptual data relationships for laboratory management. 11

5.1 Intended Excel import process. Confirm validation and storage behavior in the implementation. 12

6.1 Application dashboard (replace with an actual screenshot). 16

6.2 Excel import screen (replace with an actual screenshot). 17

Chapter 1

Introduction

1.1 Background

Laboratory-based courses involve practical sessions, group activities, attendance records, assessment, feedback, and supporting documents. Teachers may need to maintain these records across spreadsheets, registers, and separate files. As the number of students and activities increases, keeping information consistent across these sources becomes more difficult.

A centralized application can organize the information in one place and provide a consistent workflow for managing students, teams, groups, laboratory activities, attendance, and evaluation. The proposed system focuses on the teacher's workflow and provides a foundation for additional student-facing functionality in later versions.

1.2 Project Overview

The Smart Lab Management and Continuous Evaluation System is a Flutter application with Firebase/Cloud Firestore integration. Its project structure contains screens related to dashboard, students, teams, groups, lab activities, attendance, evaluation, reports, and Excel import, together with a Firestore service and session-related services. The application is organized into a user-interface layer, service layer, and data-model layer.

1.3 Motivation

The project is motivated by the need to reduce fragmented record keeping and repeated manual data entry. A consistent digital workflow can make records easier to access, simplify routine administration, and provide a base for future automation. Excel import is particularly relevant because student and team information may already be maintained in spreadsheet form.

1.4 Report Organization

Chapter 2 explains the problem and objectives. Chapter 3 describes scope and requirements. Chapter 4 presents the architecture and data model. Chapter 5 discusses the technology stack and module design. Chapter 6 describes the implementation structure. Chapter 7 outlines verification and test cases. Chapter 8 discusses limitations and future enhancements. Chapter 9 concludes the report.

Chapter 2

Problem Statement and Objectives

2.1 Problem Statement

Laboratory management information is often spread across multiple spreadsheets, registers, and documents. Student information, team allocation, attendance, laboratory activities, marks, feedback, and reports may be maintained independently. This separation can lead to duplicated data, repeated updates, increased administrative effort, and difficulty retrieving a complete history for a student or team.

The problem addressed by this project is the lack of a single, structured workflow for managing laboratory records and continuous evaluation. The system aims to bring the principal teacher-side tasks into a centralized application and support importing existing student/team data from Excel.

2.2 Problems in the Existing Approach

- **Fragmented records:** Information is distributed across multiple files and locations.
- **Repeated data entry:** Student and team information may need to be entered more than once.
- **Update inconsistency:** Changes may not be reflected in every copy of a record.
- **Attendance administration:** Session-wise attendance can be difficult to consolidate manually.
- **Evaluation tracking:** Marks and feedback may be separated from activity and student records.
- **Report preparation:** Preparing a consolidated view can require manual collection and calculation.

2.3 Aim

To develop a centralized application that supports the organization and management of laboratory-related academic records and continuous evaluation.

2.4 Objectives

1. Centralize student, team, group, and laboratory activity information.
2. Reduce repetitive data entry by supporting Excel-based import.
3. Provide structured workflows for attendance recording and evaluation.
4. Maintain marks and feedback in association with relevant academic records.
5. Make relevant records easier for teachers to retrieve and review.
6. Use a modular application structure that can support future enhancements.

2.5 Proposed Solution

The proposed solution is a teacher-oriented digital platform. The teacher accesses the application interface, manages student/team/group information, organizes laboratory activities, records attendance, enters evaluation information, and views reports. Excel import provides a route for bringing existing structured records into the application. Firebase/Cloud Firestore is used as the cloud data layer in the project structure.

Chapter 3

Scope and Requirements

3.1 Project Scope

3.1.1 Current Project Focus

The project focuses on teacher-side laboratory administration. Its source structure includes screens and services associated with the following areas: student management, team management, group management, laboratory activities, attendance, evaluation, reports, and Excel import. The completeness of each workflow should be confirmed by running the application and reviewing the relevant code before the report is submitted.

3.1.2 Out of Scope for the Current Version

The following items are treated as future enhancements unless a working implementation is confirmed: a dedicated student portal, end-to-end online submission management, automated notifications, QR-based attendance, advanced analytics, and AI-assisted evaluation.

3.2 Stakeholders

Table 3.1: Project stakeholders and interests

Stakeholder	Interest / responsibility
Teacher	Maintains student/team information, laboratory activities, attendance, evaluation, and reports.
Student	Potential future user who may view activities, attendance, marks, feedback, and submit work.
Project team	Designs, implements, documents, and maintains the application.
Institution / department	Requires organized academic records and appropriate access control.

3.3 Functional Requirements

Table 3.2: Functional requirements

ID	**Requirement**
FR-01	The system should provide an interface through which the teacher can access laboratory-management functions.
FR-02	The system should support viewing and managing student records.
FR-03	The system should support organizing students into teams and groups.
FR-04	The system should support creating or managing laboratory activity records.
FR-05	The system should support recording and retrieving laboratory attendance.
FR-06	The system should support recording evaluation marks and feedback.
FR-07	The system should provide access to consolidated reports or report views.

ID	Requirement
FR-08	The system should provide an Excel-import workflow for structured student/team data, subject to validation and supported file formats.
FR-09	The system should store and retrieve supported records through its configured backend service.

3.4 Non-Functional Requirements

Table 3.3: Non-functional requirements

Requirement	Description
Usability	Screens and workflows should be understandable to teachers with ordinary application experience.
Maintainability	Separation of screens, models, and services should help developers update individual parts.
Reliability	Data operations should report failures clearly and avoid silent loss or inconsistent updates.
Security	Access to academic information should be restricted to authorized users and suitable roles.
Performance	Common lists and record views should load within a reasonable time for the expected class size.
Scalability	Data organization should allow the system to grow to additional classes, students, and activities.

3.5 Assumptions and Constraints

- The application has access to a configured Firebase project and an internet connection for cloud operations.
- Imported spreadsheet data follows a documented column format.
- The teacher is responsible for checking the correctness of imported academic records.

- Data validation, authentication rules, and database security rules must be reviewed before real academic data is used.
- This report does not assume that future integrations are already implemented.

Chapter 4

System Design

4.1 High-Level Architecture

The application can be understood as three main software layers: the presentation layer, the application/service layer, and the cloud data layer. Flutter screens provide the user interface. Services coordinate operations such as session handling and Firestore access. Firebase/Cloud Firestore provides the configured cloud persistence layer. Excel import enters through the application and service workflow before the resulting records are stored.

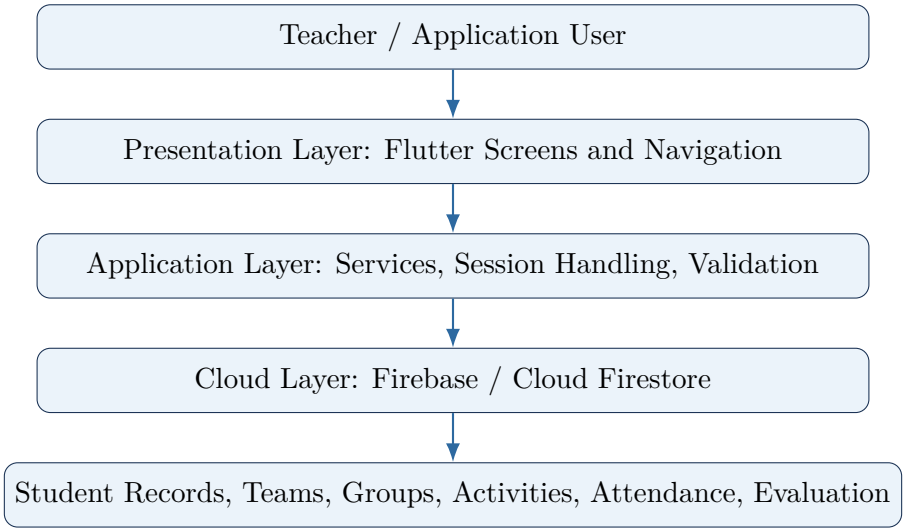


Figure 4.1: Conceptual architecture of the application. Confirm exact call paths against the implementation.

4.2 Application Workflow

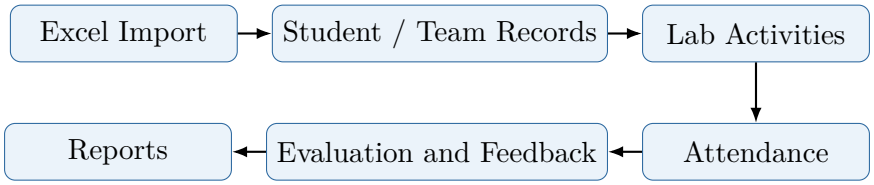


Figure 4.2: Conceptual teacher workflow. The exact sequence may vary by screen and implementation.

4.3 Major Components

Table 4.1: Major application components

Component	Responsibility
Screens	Display information and collect user actions.
Models	Define structured representations of application entities where implemented.
Services	Encapsulate data access and related operations.
Excel import	Accept spreadsheet data and map supported columns to application records.
Firestore integration	Read and write application records using the configured Firebase setup.
Session services	Maintain application or batch/session context where required.

4.4 Conceptual Data Model

The following conceptual entities describe the domain. They are not a claim that the Firestore database uses relational tables or that every relationship has already been implemented exactly as shown.

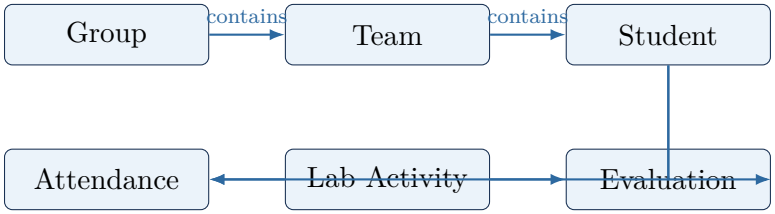


Figure 4.3: Conceptual data relationships for laboratory management.

4.5 Data Entities

Table 4.2: Conceptual entities and example attributes

Entity	Example attributes / purpose
Student	Student identifier, name, roll number, email, and team association.
Team	Team identifier, team name, and group association.
Group	Group identifier and group name.
Lab Activity	Activity identifier, title, description, and relevant date/details.
Attendance	Student/team association, activity/session association, and attendance status.
Evaluation	Student/team or submission association, score, feedback, and evaluation date where applicable.
Teacher	Teacher identity and permissions, depending on the authentication design.

Chapter 5

Technology Stack and Module Design

5.1 Flutter and Dart

Flutter is used to build the application's interface, while Dart is the programming language used for the Flutter application. A shared codebase can support a consistent interface across supported platforms, depending on configuration and deployment.

5.2 Firebase and Cloud Firestore

Firebase is used as part of the project's backend setup, and Cloud Firestore is the configured database service. Firestore is a document-oriented NoSQL database, so data is stored as documents organized within collections rather than as conventional relational tables. Appropriate document structure, access rules, validation, and query design are important for correctness and security.

5.3 Excel-Based Data Import

The Excel import module is intended to help teachers reuse structured student/team data. Example fields include roll number, name, email, team name, and group. A robust import workflow should validate required columns and values, identify malformed rows, and provide a clear summary before or after writing records. The exact validation behavior should be confirmed in the current code.

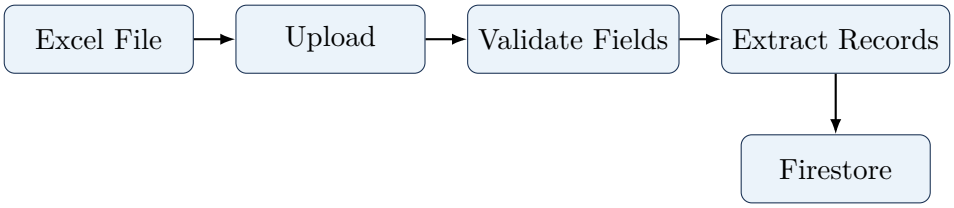


Figure 5.1: Intended Excel import process. Confirm validation and storage behavior in the implementation.

5.4 Module Descriptions

Table 5.1: Project modules

Module	Purpose
Dashboard	Intended overview of relevant laboratory records and navigation. The dashboard file should be checked because the supplied archive includes an empty dashboard screen file.
Student Management	Organizes student details and provides access to student records.
Team Management	Organizes team information and its student associations.
Group Management	Organizes groups or subgroups used in laboratory administration.
Lab Activity Management	Manages information associated with laboratory activities.
Attendance	Provides a screen/workflow for attendance records.
Continuous Evaluation	Provides a screen/workflow for marks and feedback.
Reports	Provides report-oriented views of relevant records.
Excel Import	Supports spreadsheet-based data ingestion.

5.5 Software Organization

The project follows a modular file organization with directories for screens, models, and services. In general, screens should handle presentation and user interaction, models should represent structured data, and services should isolate operations such as database reads and writes. Keeping these responsibilities separate improves readability and makes future maintenance easier.

```
lib/
  main.dart
  firebase_options.dart
  models/
```

```
  screens/
  services/
    app_session.dart
    batch_session.dart
    firestore_service.dart
```

This is a simplified view of the supplied project structure. The actual archive also contains screen files for students, teams, groups, lab activities, attendance, evaluation, reports, and Excel import. Some files in the archive are empty, so their intended role should not be mistaken for a complete implementation.

Chapter 6

Implementation Overview

6.1 Application Entry Point and Configuration

The Flutter application includes a main application file and Firebase configuration. The configuration connects the application to the selected Firebase project. Configuration values and access credentials must be handled carefully, and production security must be enforced through Firebase security rules rather than relying only on the interface.

6.2 Service Layer

The project contains a Firestore service file and session-related service files. A service layer is useful for separating data operations from the presentation layer. This organization can reduce repeated database logic across screens and provide a central location for error handling and query logic.

6.3 Record Management

The application structure includes modules for students, teams, groups, laboratory activities, attendance, evaluation, and reports. For each module, the implementation should consistently handle input validation, loading states, success/failure messages, and database errors. The exact completion status of each operation should be confirmed by inspecting the running application.

6.4 Excel Import Flow

The intended import flow is:

1. The teacher selects an Excel file.
2. The application reads the supported columns.

3. The data is validated against required fields and formats.
4. Valid rows are mapped to student/team/group records.
5. The records are written to the configured data store.
6. The application reports successful imports and any rejected rows.

The steps above describe a recommended workflow. Before submission, verify which steps are currently implemented in the Excel import screen and associated services.

6.5 Attendance and Evaluation

Attendance and evaluation are important parts of the project domain. Attendance records should be associated with the relevant student or team and laboratory session. Evaluation records should store the applicable score and feedback. If attendance is intended to influence marks, that rule must be explicitly defined by the course and implemented transparently; it should not be assumed from the existence of separate attendance and evaluation screens.

6.6 Screenshots

Add screenshots from the actual running application to demonstrate the implementation. Replace the boxes below with real screenshots using `\includegraphics` once the images are available in Overleaf.

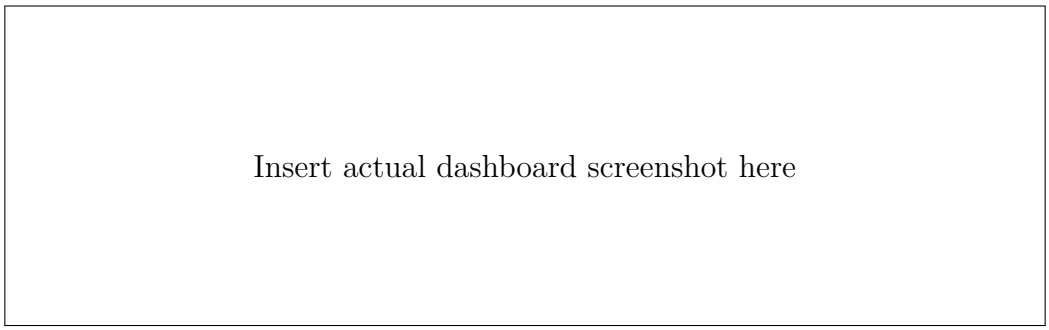


Figure 6.1: Application dashboard (replace with an actual screenshot).

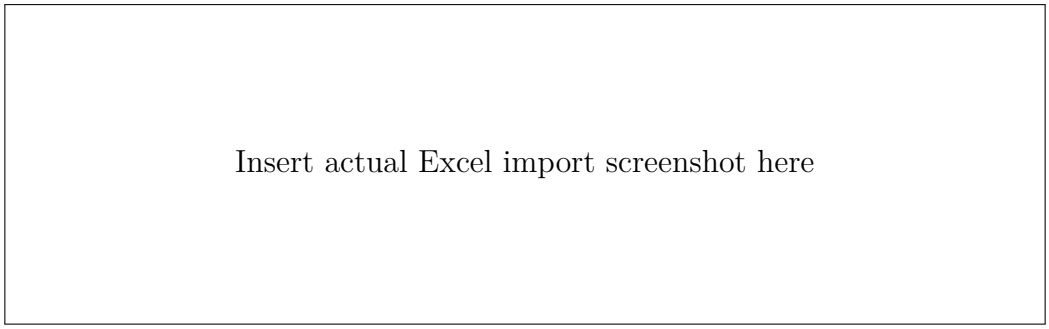


Figure 6.2: Excel import screen (replace with an actual screenshot).

Chapter 7

Verification and Test Plan

7.1 Purpose of Testing

Testing is required to determine whether each workflow behaves as expected and whether the application handles invalid input and backend errors appropriately. The following table is a proposed test plan, not a claim that every test has already been executed or passed. Record actual outcomes after running the application.

Table 7.1: Proposed test cases

ID	Test	Expected result	Actual status
TC-01	Import a valid Excel file	Supported rows are parsed and the expected records are created.	To be tested
TC-02	Import a file with missing required columns	The application explains the validation problem and avoids incorrect imports.	To be tested
TC-03	Import a row with invalid field values	The row is rejected or flagged according to the defined validation rules.	To be tested
TC-04	Create or update a student	The correct student record is displayed and persisted.	To be tested
TC-05	Create or update a team/group	The team/group record and associations are consistent.	To be tested
TC-06	Record attendance	The selected attendance status is stored and retrieved correctly.	To be tested

ID	Test	Expected result	Actual status
TC-07	Enter evaluation and feedback	The correct score and feedback are associated with the intended record.	To be tested
TC-08	Retrieve report data	The report view displays the expected records and values.	To be tested
TC-09	Simulate a backend failure	The application displays a suitable error and does not silently report success.	To be tested
TC-10	Check unauthorized access	Access to restricted records or operations is denied by the configured security rules.	To be tested

7.2 Testing Categories

- **Functional testing:** Verifies individual actions such as importing records, recording attendance, and entering evaluation data.
- **Integration testing:** Verifies interaction between screens, services, and Firestore.
- **System testing:** Verifies the end-to-end workflow across multiple modules.
- **Usability testing:** Checks whether teachers can complete common tasks with clear feedback.
- **Security testing:** Checks authentication, authorization, and Firestore rules using test accounts and non-production data.

7.3 Evidence to Include

For the final submission, record the date, environment, test data, actual result, and pass/fail status for each executed test. Include screenshots of successful workflows and validation errors where appropriate. Do not report a test as passed until it has actually been executed.

Chapter 8

Current Limitations and Future Enhancements

8.1 Current Limitations

The limitations below describe areas to verify or improve in the current project. Keep only the statements that accurately reflect the final running version.

1. **Teacher-centric workflow:** The current project is primarily organized around teacher-side management. A dedicated student portal is a possible extension.
2. **Submission management:** End-to-end upload, deadline, versioning, and submission-tracking workflows may require further development.
3. **Attendance automation:** Attendance is designed around teacher-side recording unless another method has been implemented and verified.
4. **Advanced analytics:** Trend analysis, class comparisons, and early-warning dashboards can be added in a later version.
5. **Automated notifications:** Reminders for activities, submissions, and evaluation updates are potential future functionality.
6. **AI-assisted evaluation:** AI-based analysis and feedback suggestions are future enhancements and should not replace teacher judgement.

8.2 Future Enhancements

8.2.1 Student Portal

Introduce student authentication and role-appropriate views for profile details, teams, laboratory activities, attendance, marks, and feedback.

8.2.2 Online Submission Management

Allow students to upload reports, presentations, source code, or other required files. Store file metadata with the associated student/team and activity, and use a suitable file-storage service for the actual files.

8.2.3 QR-Based Attendance

A controlled QR workflow could help record attendance. The design would need safeguards against repeated scans, sharing of QR codes, and unauthorized attendance changes.

8.2.4 Analytics Dashboard

Provide summary information such as attendance percentages, activity completion, evaluation distributions, and team-level progress. Calculations should be clearly defined and checked against source records.

8.2.5 Notifications

Add reminders for upcoming laboratory activities, submission deadlines, and published feedback. Notifications should respect user preferences and institutional policies.

8.2.6 AI-Assisted Evaluation

A future AI service could identify missing report sections or provide preliminary feedback. The teacher should review all suggestions and retain responsibility for final marks. Student data should be handled according to institutional privacy requirements.

8.2.7 Access Control and Security

Define roles such as administrator, teacher, and student, and enforce permissions through authentication and backend security rules. Client-side hiding of controls alone is not sufficient authorization.

Chapter 9

Conclusion

The Smart Lab Management and Continuous Evaluation System addresses the administrative complexity of managing laboratory-related academic records across separate files and manual processes. The project uses Flutter and Dart for the application interface and Firebase/Cloud Firestore for cloud data management. Its modular structure includes screens and services associated with students, teams, groups, laboratory activities, attendance, evaluation, reports, and Excel import.

The project provides a foundation for organizing laboratory workflows in a centralized application. Excel-based import is intended to reduce repetitive data entry, while the attendance and evaluation modules focus on keeping relevant academic records accessible. The final quality and completeness of these workflows must be demonstrated through execution evidence and test results.

Future work can expand the system with student-facing access, online submissions, file storage, notifications, analytics, QR-based attendance, stronger role-based access control, and AI-assisted feedback. With careful validation, security, and testing, the platform can evolve into a more comprehensive laboratory management solution.

Bibliography

[1] Flutter Documentation, *Flutter Documentation*. Available at: https://docs.flutter.dev/.

[2] Dart Documentation, *Dart Programming Language*. Available at: https://dart.dev/guides.

[3] Firebase Documentation, *Firebase Documentation*. Available at: https://firebase.google.com/docs.

[4] Firebase Documentation, *Cloud Firestore*. Available at: https://firebase.google.com/docs/firestore.

[5] Overleaf, *Learn LaTeX*. Available at: https://www.overleaf.com/learn.

Appendix A

Final Submission Checklist

Before submitting this report, the team should:

- Replace all institution, faculty, course, and academic-session placeholders.
- Confirm each module's implementation status against the running application.
- Replace screenshot placeholders with actual application screenshots.
- Execute the proposed test cases and update the actual-status column.
- Confirm the Firestore data structure and security rules used in the project.
- Remove or revise any statement that does not match the final code.
- Add any required course-specific sections, formatting, or declarations.